RAG Assignment Report

Baseline Retrieval vs. Cross-Encoder Re-Ranking

Course: LMA **Name:** Anurag Kaushal
Roll Number: 2025202013 **Repository:** `https://github.com/Anurag-1205/rag_assign1`

Purpose. In this experiment, I compare a dense-retrieval baseline with a two-stage pipeline that re-ranks the same retrieved candidates using a cross-encoder. I used a 39-page legal corpus containing the *Digital Personal Data Protection Act, 2023* and the *Digital Personal Data Protection Rules, 2025*. It is a useful test document because the answer is often a short qualification hidden beside a broad rule—for example, an exception, a prohibition, or a role-specific amount.

1. Experiment Setup

The PDF was cleaned and divided into 230 chunks of 120 words with a 30-word overlap. I kept the corpus, initial top-10 retrieval, Gemini model, and prompt unchanged in both pipelines. This makes the comparison fair: if the final answer changes, I can trace that change back to which three chunks the LLM received.

Table 1: System configuration

Component	Configuration
First-stage retriever	`all-MiniLM-L6-v2` bi-encoder with L2-normalised embeddings
Vector search	FAISS `IndexFlatIP`; inner product on normalised vectors equals cosine similarity
Candidates retrieved	Top 10 chunks per query
Pipeline 1 context	Top 3 chunks by bi-encoder score
Pipeline 2 context	Same 10 candidates, re-ranked by `ms-marco-MiniLM-L-6-v2`; top 3 used
Answer model	Gemini `gemini-3.7-flash`, temperature 0; identical prompt in both pipelines
Evaluation	112 difficult queries covering distractors, exceptions, and negation

2. What the Two Stages Do

A **bi-encoder** represents the query and chunks separately, then finds similar chunk vectors quickly. I think of it as the fast first reader: it can identify passages about the same topic, but it may not notice the one word that changes the legal meaning. For example, a query about the Chairperson's salary can be pulled towards passages about other Members' salary because the topic is very similar.

A **cross-encoder** reads the query and a candidate chunk together. I use it as a second, more careful reader: it can compare the wording of the question directly with the passage and pay attention to details such as "not entitled", "except", and specific roles. It is more precise, but must run once per query–chunk pair, so I apply it only to the ten first-stage candidates.

3. Results: Retrieval Quality and Answer Impact

3.1 Aggregate results

Table 2: Gold-chunk retrieval performance across 112 stress-test queries

Metric	Baseline	Re-ranked	Change
Hit@1	0.393	0.446	+0.053
Hit@3	0.518	0.589	+0.071
Hit@10	0.696	0.696	0.000
MRR@10	0.473	0.533	+0.060

I focus mainly on Hit@3 because the LLM sees only three chunks. Re-ranking moved the gold evidence into that context for 11 queries. Hit@10 stays unchanged because the cross-encoder only re-orders the original ten candidates; it does not retrieve new ones. In other words, re-ranking improves the shortlist, not the first-stage recall.

Table 3: What happened to the gold chunk after re-ranking

Outcome	Queries
Already in baseline top 3	55
Promoted into top 3 (fixed)	11
Not in the initial top 10	34
Still outside top 3	9
Moved down (worse)	3
Total	**112**

3.2 Representative rank shifts

The first six examples below are successful rescues: the key clause entered the top three. EXC-20 is included to show that re-ranking does not improve every query.

Case	Rank shift	Observed answer impact
DIS-15: penalty	$7 \rightarrow 1$	Baseline said the exact Schedule rule was absent. Re-ranked answer gave the correct "up to the extent applicable" rule.
DIS-05: salary	$6 \rightarrow 2$	Baseline gave the wrong amount (Rs 4,00,000). Re-ranked answer correctly gave Rs 4,50,000 per month.
NEG-10: sitting fee	$5 \rightarrow 1$	Baseline said the excerpts did not say. Re-ranked answer correctly stated that Members are not entitled to a sitting fee.
NEG-06: data access	$5 \rightarrow 3$	Baseline had no answer. Re-ranked answer correctly stated that a Consent Manager must not be able to read the data.
DIS-18: appointment	$9 \rightarrow 3$	Baseline could not identify who appoints the Chairperson. Re-ranked answer correctly identified the Central Government.
EXC-63: court exemption	$4 \rightarrow 2$	Baseline lacked the exception. Re-ranked answer identified the qualified exemption for judicial or quasi-judicial processing.

Case	Rank shift	Observed answer impact
EXC-20: outsourcing	$1 \rightarrow 7$	Counterexample: correct baseline evidence dropped out of the top three, so the re-ranked answer became insufficient.

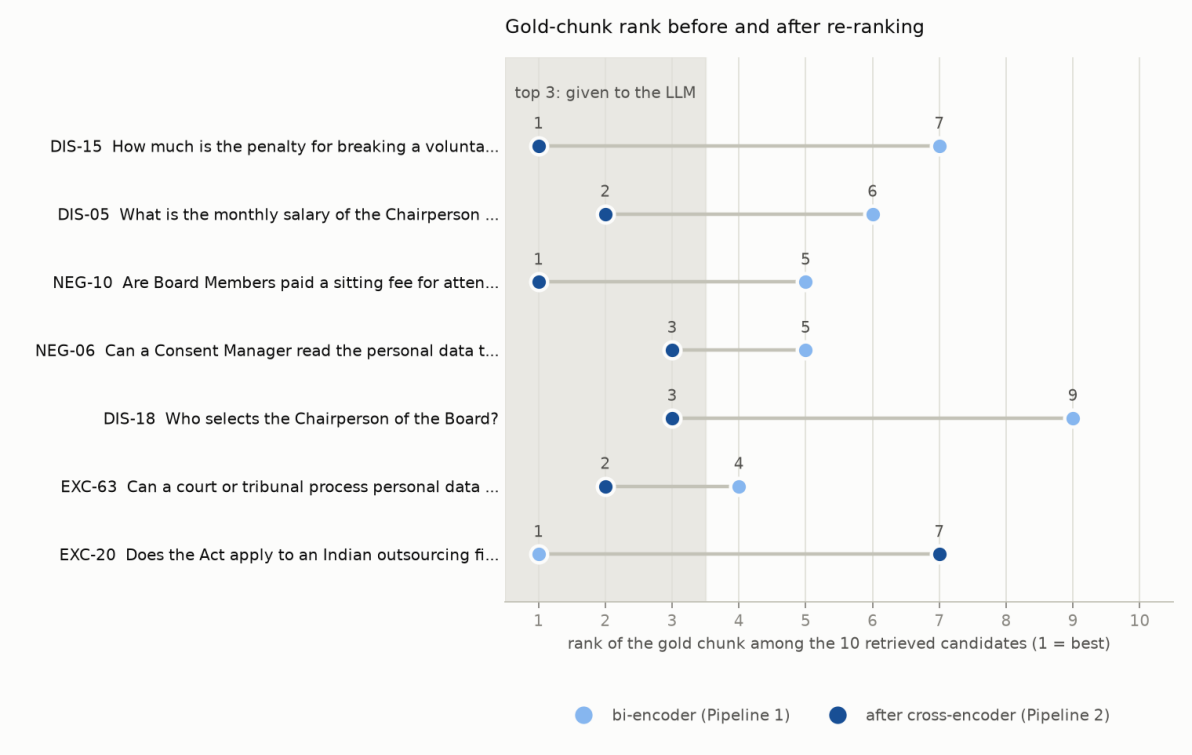


Figure 1: Rank of the gold chunk before and after re-ranking. Ranks 1–3 are passed to the LLM.

These are stress-test queries, not average production traffic. I selected them because they make the failure modes easy to see. When a relevant clause moves into the top three, an answer can change from missing or wrong to correct. DIS-05 is the clearest example: the selected context changed the salary from the incorrect Rs 4,00,000 to the correct Rs 4,50,000.

4. Latency Cost

Latency was measured on CPU over 180 timed runs: six rescued queries repeated 30 times after warm-up.

Table 5: Retrieval latency

Stage	Median (ms)	p95 (ms)
Query embedding	12.50	14.26
FAISS search (top 10)	0.07	0.12
Pipeline 1 total	**12.58**	**14.40**
Cross-encoder scoring (10 pairs)	361.46	384.00
Pipeline 2 total	**374.40**	**395.89**

The cross-encoder causes almost all of the extra 361.8 ms. So, in this small experiment,

I get better ordering at roughly 30 times the retrieval-stage cost. The Gemini call still dominates end-to-end time (about 7.24 seconds in either pipeline), but the retrieval difference would matter more in a high-throughput system or with a faster local answer model.

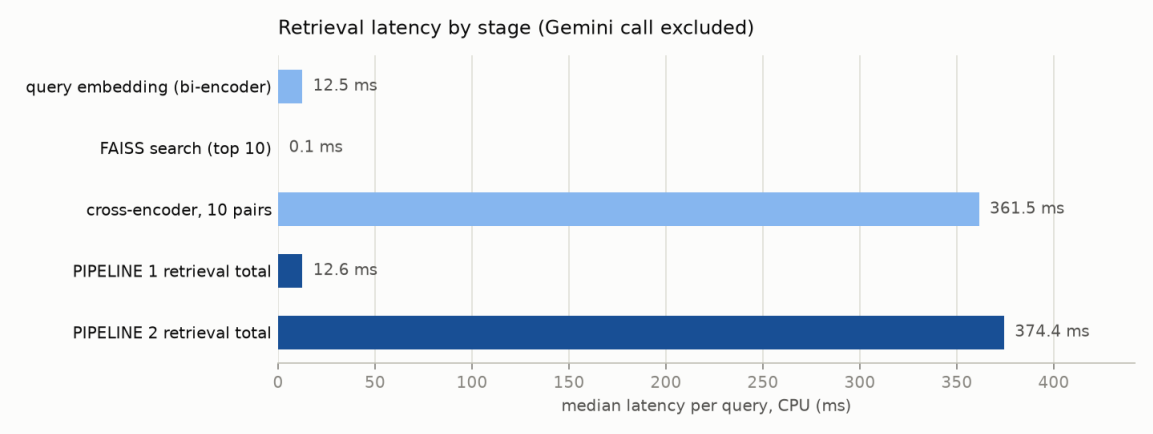


Figure 2: Median retrieval latency by stage; Gemini-generation time is excluded.

5. Why Not Re-rank Every Chunk?

A cross-encoder cannot reuse a stored document vector: it must jointly process the query with each chunk. On this CPU, scoring all 230 chunks took 9.75 seconds (about 42.4 ms per pair), compared with a 0.07 ms FAISS search over the retrieved set. The table gives linear extrapolations from the measured per-pair cost.

Table 6: Estimated exhaustive cross-encoder time per query

Collection size	Estimated time
230 chunks	9.8 s
10,000 chunks	424.0 s (about 7.1 min)
100,000 chunks	4,240.4 s (about 70.7 min)
1,000,000 chunks	42,403.5 s (about 11.8 h)

This is why I use a two-stage design: the bi-encoder gives fast recall over the full collection, while the cross-encoder spends its compute budget only on a short candidate list. The remaining weakness is clear: 34 gold chunks were not in the initial top 10, so re-ranking had no chance to recover them.

6. Conclusion

Cross-encoder re-ranking improved Hit@3 by 7.1 percentage points and corrected concrete answer failures, including an incorrect salary, missing prohibitions, and missed legal exceptions. The trade-off is clear: median retrieval latency rose from 12.58 ms to 374.40 ms, and three queries were made worse by re-ranking.

My main conclusion is not that re-ranking replaces retrieval. It works best with a strong first-stage retriever: the bi-encoder finds the possible evidence quickly, and the cross-encoder checks which of those candidates actually answers the question. Evidence outside the initial top 10 remains unreachable.